Phase 4 | Interview Question Bank with Answers

Short answer, interview answer, evidence, follow-up, and common pitfalls.

FastAPI Edition | Repository-grounded study material | Snapshot: 2026-10-24

A. Project and architecture

Q: What does the project do?

Short: It creates station/date/time demand scenarios and compares them with modeled service and network assumptions.

Interview answer: “MetroInsights is a FastAPI-based planning prototype. The user selects a station, date, time, and line; the service creates an hourly estimate from a scikit-learn model trained on synthetic scenarios, calculates a daily profile, compares line demand with configurable boarding-clearance assumptions, and returns line-aware shortest-path times. The interface exposes those results. It is not live ridership prediction because the target data is synthetic.”

Evidence: `app/routes.py`, `app/forecasting.py`, `app/scheduling_optimizer.py`, `app/travel_graph.py`.

Follow-up: What is the biggest limitation? Answer: No observed passenger-count validation, so excellent holdout metrics only show performance on generated scenarios.

Common mistake: saying it predicts actual DMRC ridership accurately.

Q: Why FastAPI?

Short: It provides an ASGI API framework, typed validation options, and OpenAPI docs.

Strong answer: “The project exposes JSON endpoints and server-rendered pages. FastAPI provides Starlette request/response tools and generated API docs. I offload synchronous inference work to a threadpool so the async endpoint does not block the event loop. The current request uses raw Request, so typed Pydantic request schemas are a clear next improvement.”

Follow-up: Does async make sklearn faster? No; it protects event-loop responsiveness, while model computation itself is still synchronous.

Q: Walk through one request.

Answer: API route parses JSON/query input; helper validates date/time; calendar and service-window functions derive context; forecaster creates a schema-aligned feature row and predicts; scheduler compares demand with assumed clearance; graph runs Dijkstra; JSON response includes results and provenance. Closed hours override demand/frequency as closed. Cite `routes.py` and modules.

B. Data and ML

Q: What is the dataset?

Answer: 3,361,176 synthetic hourly records created from 243 station records, 728 dates, and 19 time slots. This is generated data, not a DMRC tap-in/tap-out feed. The station inputs are project records and topology requires source verification.

Q: How did you avoid leakage?

Answer: Split by date at 2026-10-01. Historical aggregates for training rows use earlier observations; holdout aggregates are built only from training period. I would still audit every feature's availability and validate on real held-out observations before

making accuracy claims.

Follow-up: Why not random split? Random rows from the same dates/patterns can leak temporal structure into both sets and overstate future performance.

Q: Interpret the metrics.

Answer: Synthetic holdout has MAE 26.58 passengers/hour, RMSE 51.10 passengers/hour, R^2 0.9946, MAPE 5.18%. This indicates close fit to generated labels. It is not evidence of comparable accuracy on real passenger counts. Compare to historical mean and Ridge on same holdout.

Pitfall: calling R^2 "99.46% accuracy." R^2 is not classification accuracy.

Q: Why gradient boosting?

Answer: It handles nonlinear thresholds/interactions in tabular data with little feature scaling; histogram-based split search is practical at this scale. Ridge and historical mean establish baselines. Evaluate per segment and calibrate with real targets before operational use.

Q: Why cyclic encodings?

Answer: Raw hour treats 23 and 0 as far apart, despite adjacency. Sine/cosine put a periodic value on a circle. Include both coordinates so the phase can be distinguished.

C. Mapping and routing

Q: Explain the station mapping system.

Answer: The station CSV is canonical metadata; route arrays contain ordered labels; normalization and alias rules map labels to canonical names. The graph expands interchanges into `(station,line)` nodes, adds edges between adjacent stops, and explicit transfer edges with penalties. Unmatched names are skipped today, so an audit should fail startup or report them.

Q: Why line-expanded graph nodes?

Answer: A station name alone cannot represent which line/platform a traveler occupies. Separate line states prevent a free line switch. An explicit interchange edge charges transfer time.

Q: Why Dijkstra?

Answer: All segment and transfer weights are nonnegative, and the desired output is shortest estimated time from a source to many destinations. A heap-based adjacency-list implementation is simple and efficient for this network.

Q: Are travel times accurate?

Answer: They are topology-based estimates. Adjacent weights derive from coordinates and fixed average speed where coordinates look plausible, otherwise a fallback segment time; transfers use one average penalty. No timetable, service disruptions, actual dwell or station-specific transfer walks are modeled.

D. Scheduling and semantics

Q: What does capacity utilization mean?

Answer: Estimated station boarding demand per hour divided by modeled per-hour boarding-clearance capacity under the recommended frequency. Clearance reserves capacity for through passengers. It is not live occupancy, passenger load on a particular train, nor platform density.

Q: How is train frequency recommended?

Answer: Demand is split by line; a peak/off-peak baseline headway and a through-load assumption produce per-train boarding clearance. If demand exceeds offered clearance, target frequency rises; if light in eligible off-peak windows, it may relax. Fleet and signaling bounds clamp it, then a practical headway is selected.

Pitfall: presenting config parameters as verified live DMRC timetable facts.

Q: Why can a rush-hour result still show comfortable capacity?

Answer: The demand forecast may still be low because it is synthetic, while denominator assumptions may be generous. Peak status affects the available clearance fraction, but badge is driven by utilization thresholds. Validate demand with observed data and calibrate per-line capacities before interpreting operationally.

E. API, reliability, security

Q: How are invalid inputs handled?

Answer: Date/time parse errors and malformed JSON return 400. Non-object JSON also returns 400. Unknown stations are not consistently rejected yet and can fall back to default index/info; validate station against canonical IDs and return a clear 404/422.

Q: What can fail at startup?

Answer: Missing/corrupt joblib, station CSV, operations CSV, or import errors. Because dependencies are loaded at module import, app import fails. Add explicit startup checks and diagnostics; test missing artifacts in isolated tests.

Q: Is it secure for public deployment?

Answer: It is a prototype with no auth or rate limiting. Inputs should be bounded and validated; dependencies/artifacts trusted; error responses avoid internal paths; production deployment should add security headers, TLS termination, rate limits, monitoring, and data provenance controls as appropriate.

F. Difficult follow-ups

Q: What if station names collide after normalization?

Answer: Current dictionary insertion can overwrite an earlier canonical row. Build a normalization index to lists, detect collisions, resolve them explicitly and fail data validation rather than silently choosing.

Q: Why multiple source nodes in Dijkstra?

Answer: The origin interchange is considered accessible from any of its modeled line-states at zero initial transfer cost. Once search leaves the source, changing line requires a transfer edge. This assumption means we do not charge a transfer merely to start from a chosen station.

Q: What is MAPE caveat?

Answer: Percentage errors explode near zero and cannot naturally handle zeros. This implementation excludes zero targets; report denominator policy and also inspect MAE/RMSE by demand band.

Q: Could model fit synthetic data too well?

Answer: Yes. A generator creates structured labels; the model may reconstruct its rules. Strong synthetic holdout scores test internal consistency, not external validity. Compare to observed counts and document domain shift.

G. Resume cross-examination

If claiming “3.15M”, explain this code snapshot now reports 3,361,176 records; resume should be updated to current evidence or a precisely documented earlier snapshot. If claiming 243 stations, say station records, not necessarily official unique operational station count. If claiming Dijkstra travel estimates, call them topology-based estimates. If claiming FastAPI, disclose that migration runtime smoke verification was unavailable in the offline environment until you run it locally.

How to use this series

Read Phase 1 first, then Phase 2 while opening the source files. Use Phases 3, 5, 6, and 7 to deepen technical understanding; rehearse with Phases 4 and 8; revise with Phase 9. The editable Markdown beside each PDF is the source of truth for corrections.

Evidence and honesty rules

MetroInsights is a planning prototype. Its demand targets and holdout are synthetic. Model metrics describe agreement with generated scenario labels, not real passenger counts or operational DMRC accuracy. Network topology, coordinates, capacities, headways, service frequency, and interchange splits are project data or configurable assumptions unless separately verified. Say “estimated” and explain the assumptions. Do not describe this as a live passenger-count, train occupancy, or production DMRC system.

Verified snapshot used in this edition

The cleaned station table contains 243 station records. The topology graph's destination count depends on the selected source and excludes the source itself; 242 destinations can be returned when all other modeled stations are connected. Do not say 226 or 228 without checking the current data. The project defines 11 route keys, including branch keys. The synthetic training series uses 243 stations x 728 dates x 19 hourly slots (05:00-23:00), or 3,361,176 rows. A chronological holdout begins 2026-10-01 and has 438,615 rows. The recorded HistGradientBoostingRegressor scores are MAE 26.58, RMSE 51.10, R2 0.9946, and MAPE 5.18%; Ridge MAE is 53.67 and historical station/day/hour mean MAE is 60.25. These are synthetic-only evaluation results.

Current stack and constraints

FastAPI + Uvicorn, Starlette/Jinja2, server-rendered HTML with JavaScript/CSS, Pandas, NumPy, scikit-learn, joblib, CSV/JSON, and a custom adjacency graph with heapq Dijkstra. No relational database, live ridership feed, authentication, or distributed job queue is present. The generated multi-million-row CSV is a training input; runtime inference loads compact serialized model/metadata plus station and operations inputs, rather than reading every training row on each request. Runtime startup imports routes and eagerly loads artifacts/builds the graph, so missing files can prevent startup. The current FastAPI migration has passed static compilation and diff checks in prior work, but an actual runtime smoke test was not possible in the offline environment; verify locally before claiming deployment verification.